

CMSC828G: Special Topics in Systems for Machine Learning

Transformer and Language Model

Instructor: Yafan Huang



DEPARTMENT OF
COMPUTER SCIENCE

From Statistical Language Models to Transformers

- Language modeling: predict the next token given previous tokens.

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

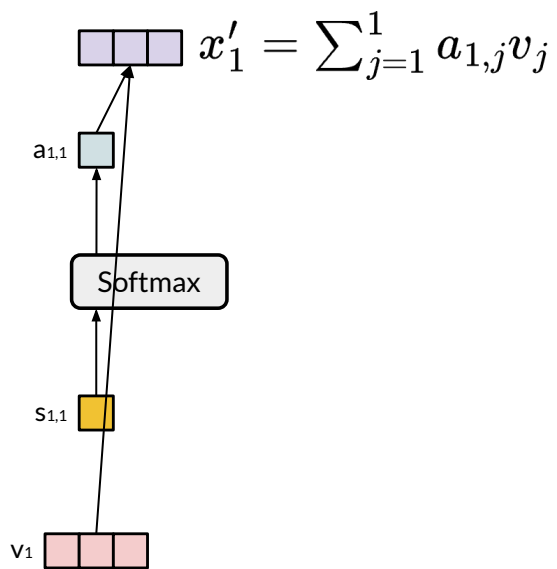
From Statistical Language Models to Transformers

- 1990s ~ 2000s – Statistical LLMs: n-gram models.
- 2000s ~ Neural Language Models: Learn word representations / embeddings.
- 2010s ~ Recurrent Neural Networks: RNN -> LSTM / GRU.
- 2014 ~ 2016 - Attention: Seq2Seq + Attention.
- 2017 ~ Transformer: “Attention is all you need”, removing recurrence.
- 2018 ~ Today - Large Language Models (LLMs): transformer + scaling.

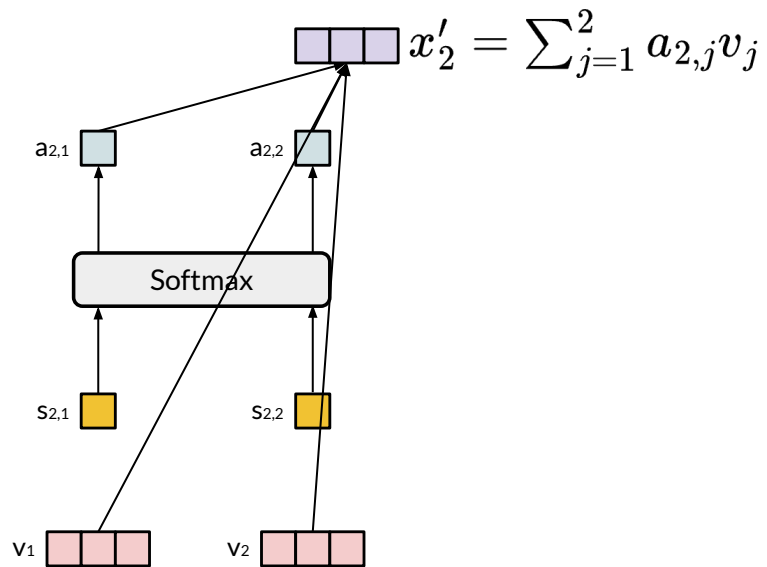
Attention

- Attention computes a weighted combination of a set of values, where the weights indicate how relevant each value is to the current output.
- Example: The cat sat ? -> The cat sat **down**

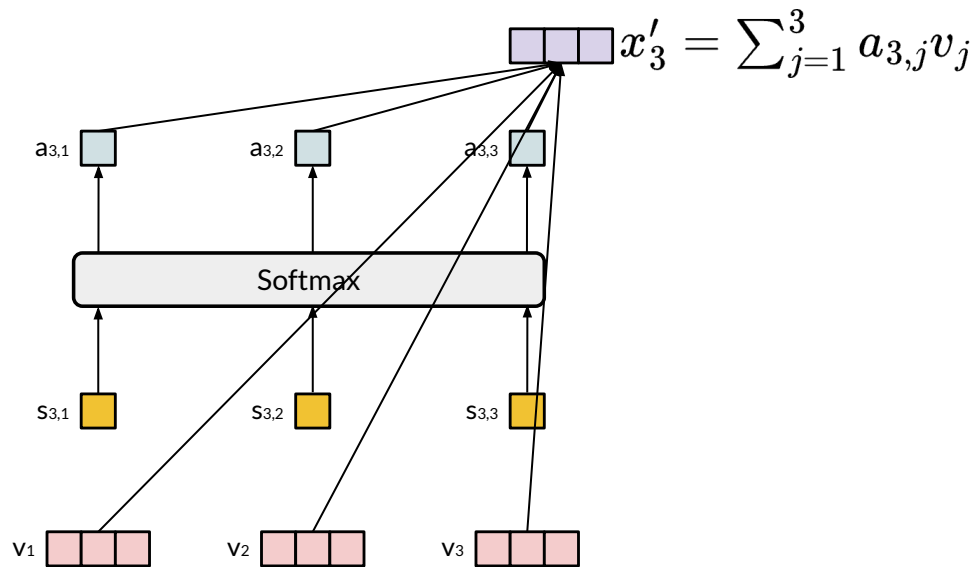
Attention



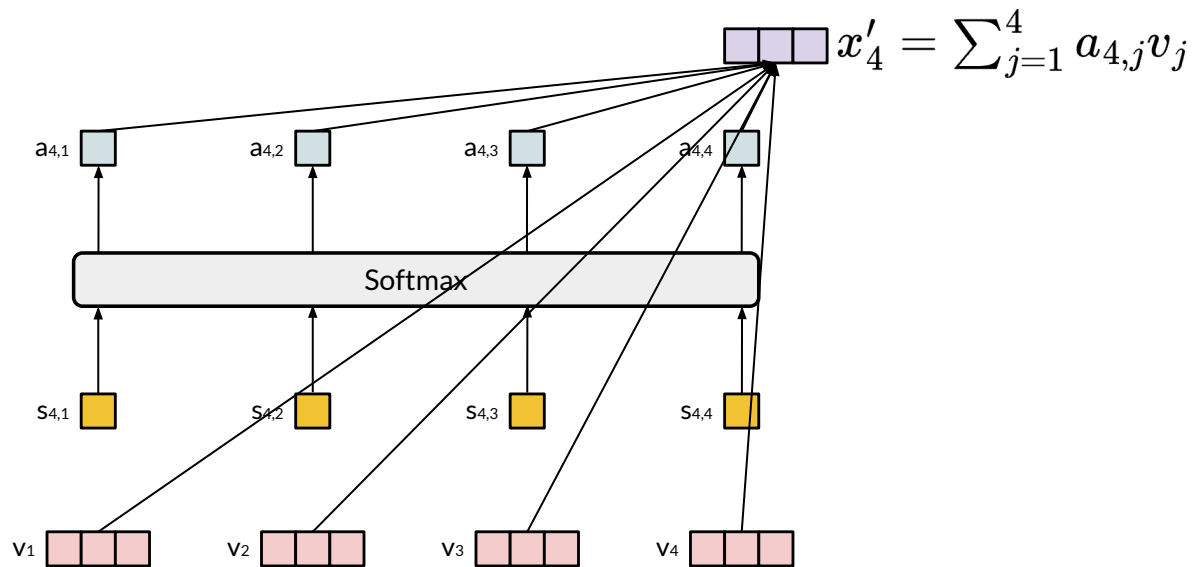
Attention



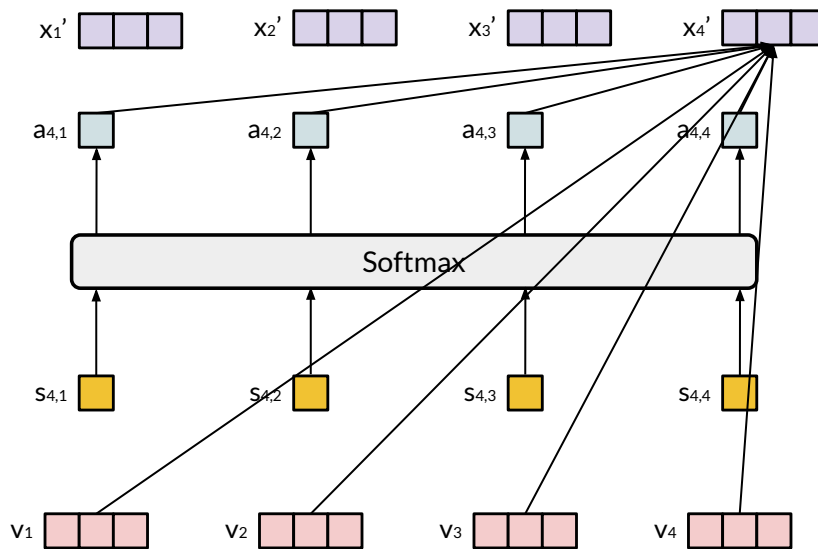
Attention



Attention



Attention



$$x'_t = \sum_{j=1}^t a_{t,j} v_j$$

attention weights

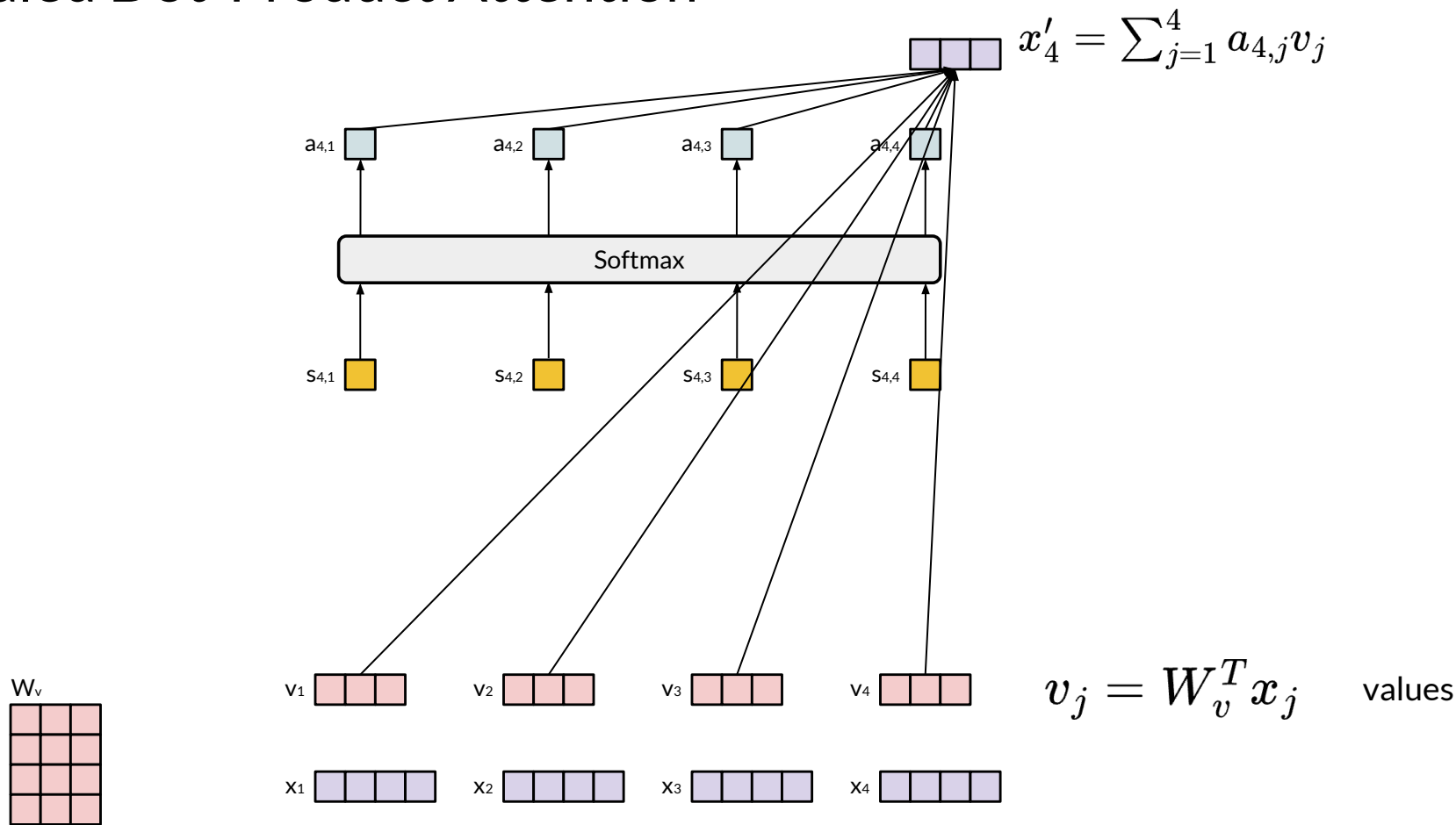
scores

values

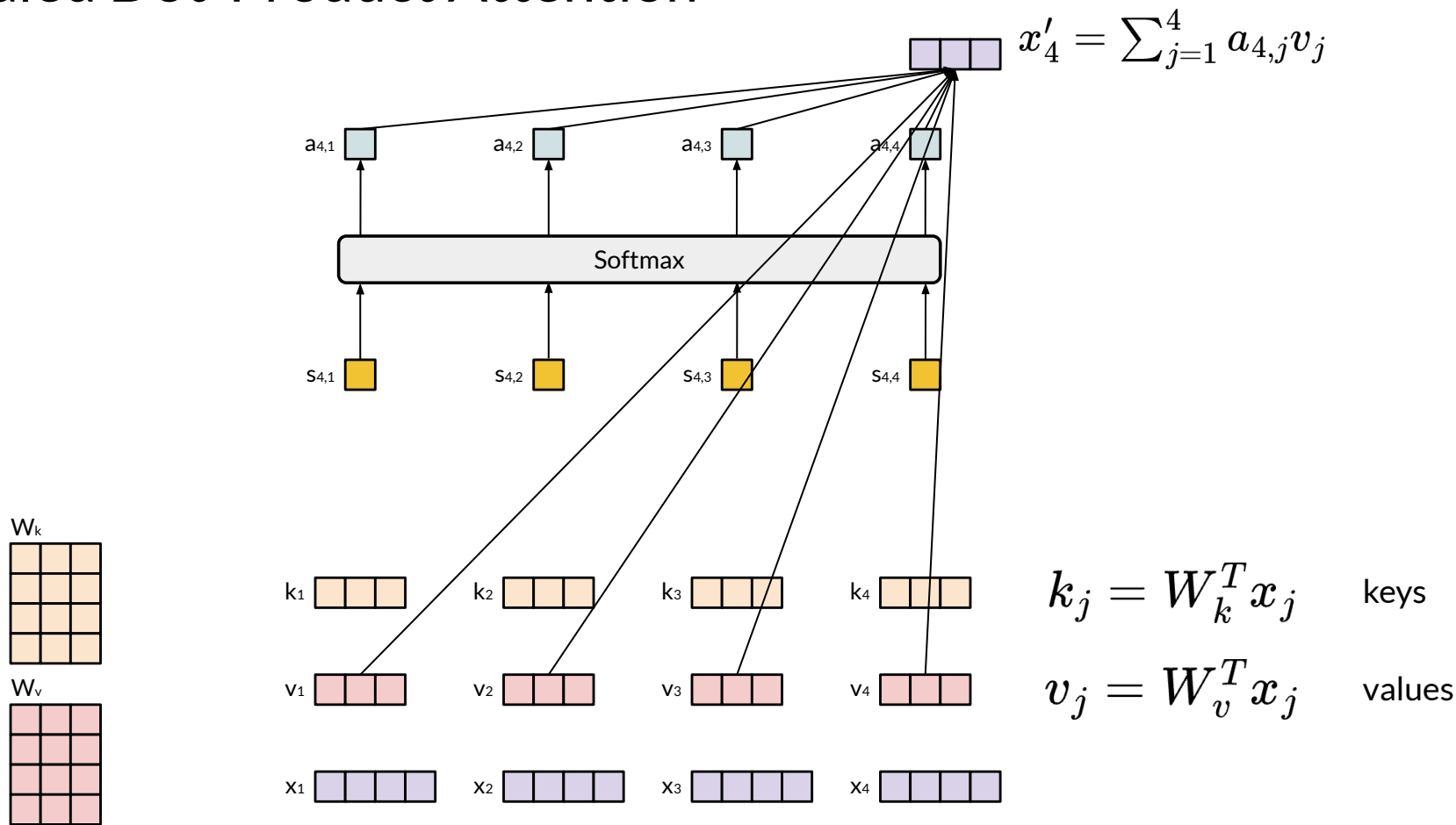
Scaled Dot-Product Attention

- Scaled dot-product attention computes attention weights by taking the scaled dot products between queries and keys, then uses these weights to combine the values.
 - **Query (Q):** What you are currently looking for.
 - **Key (K):** What each token offers.
 - **Value (V):** The actual content.

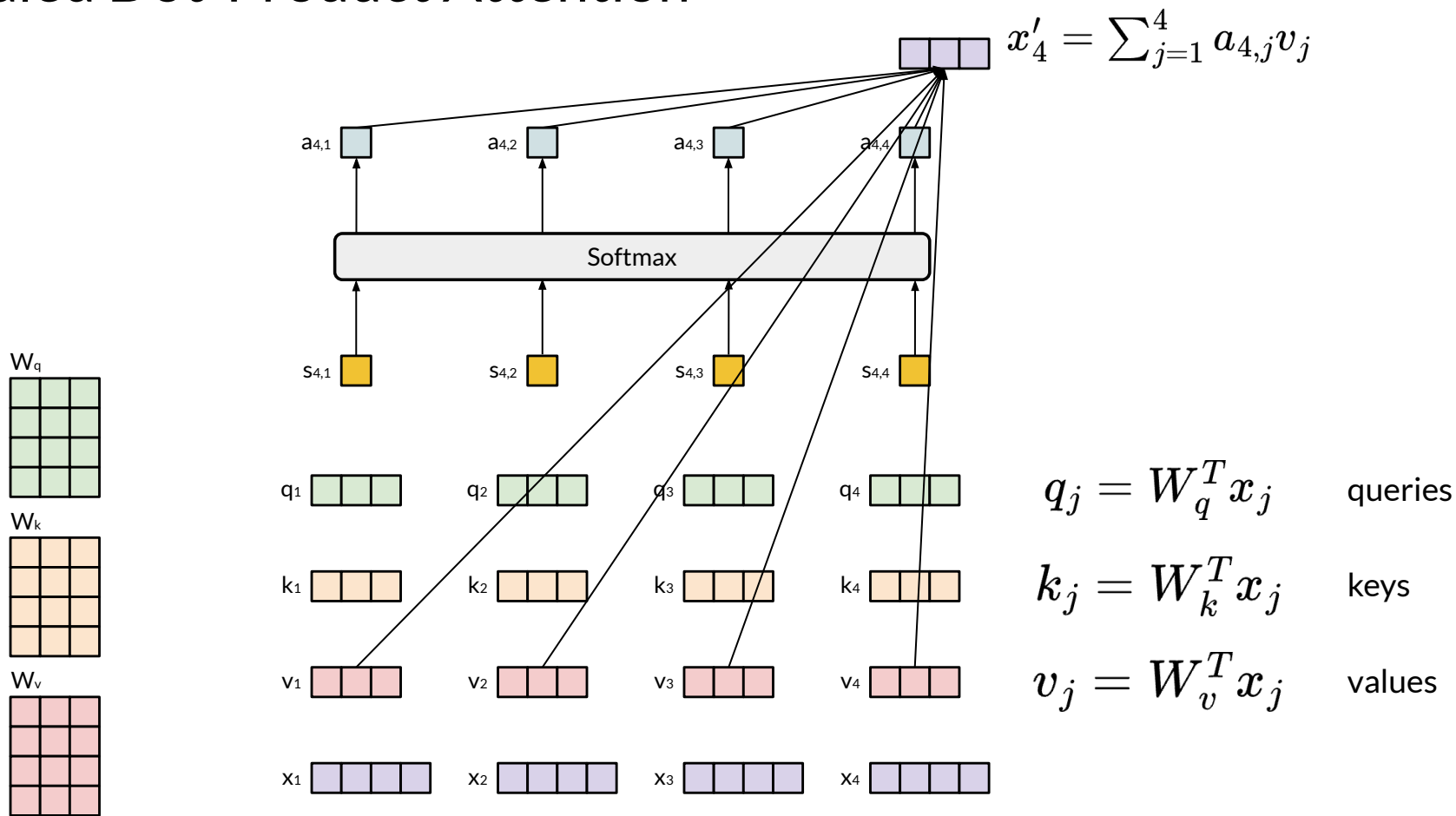
Scaled Dot-Product Attention



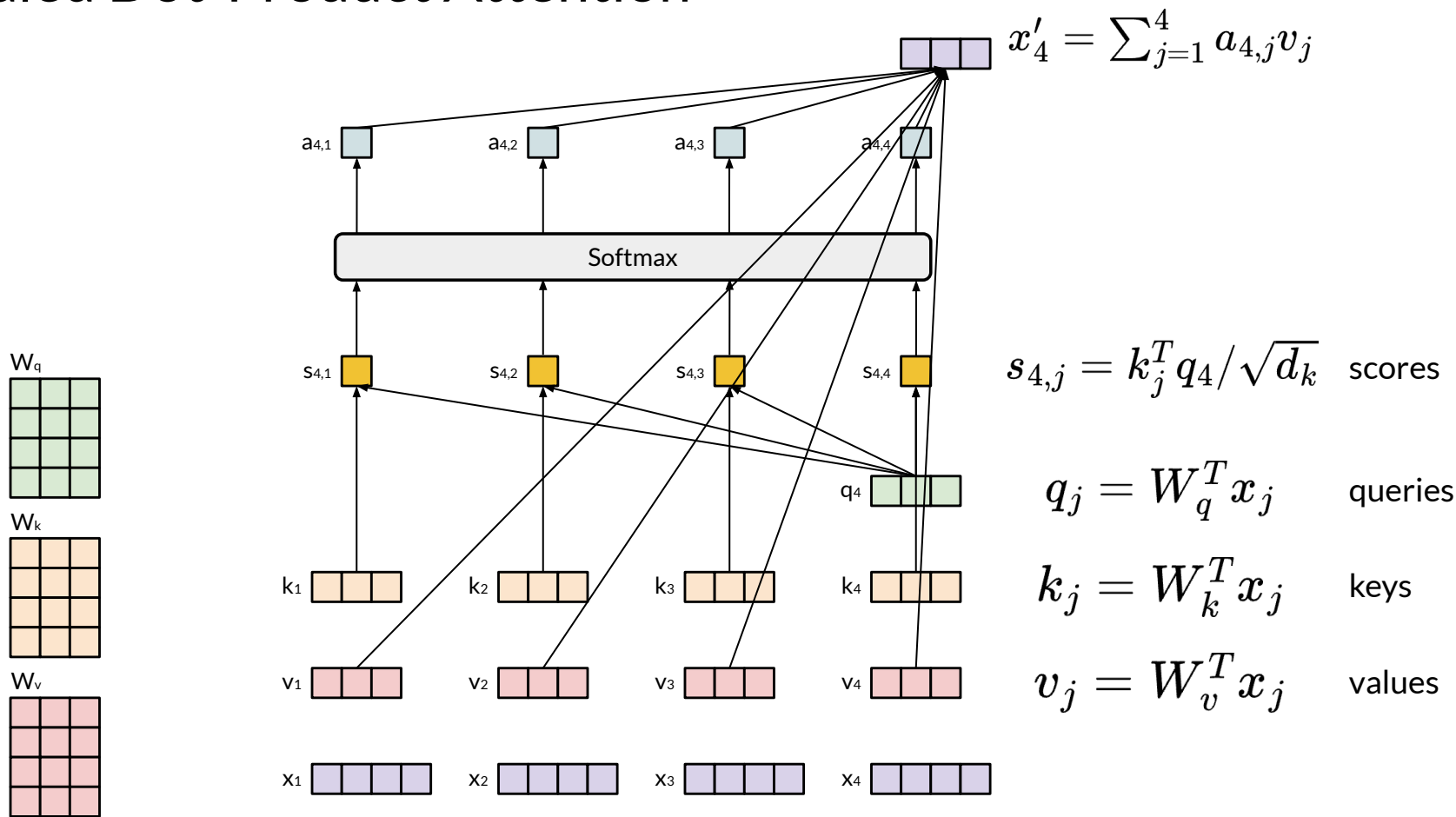
Scaled Dot-Product Attention



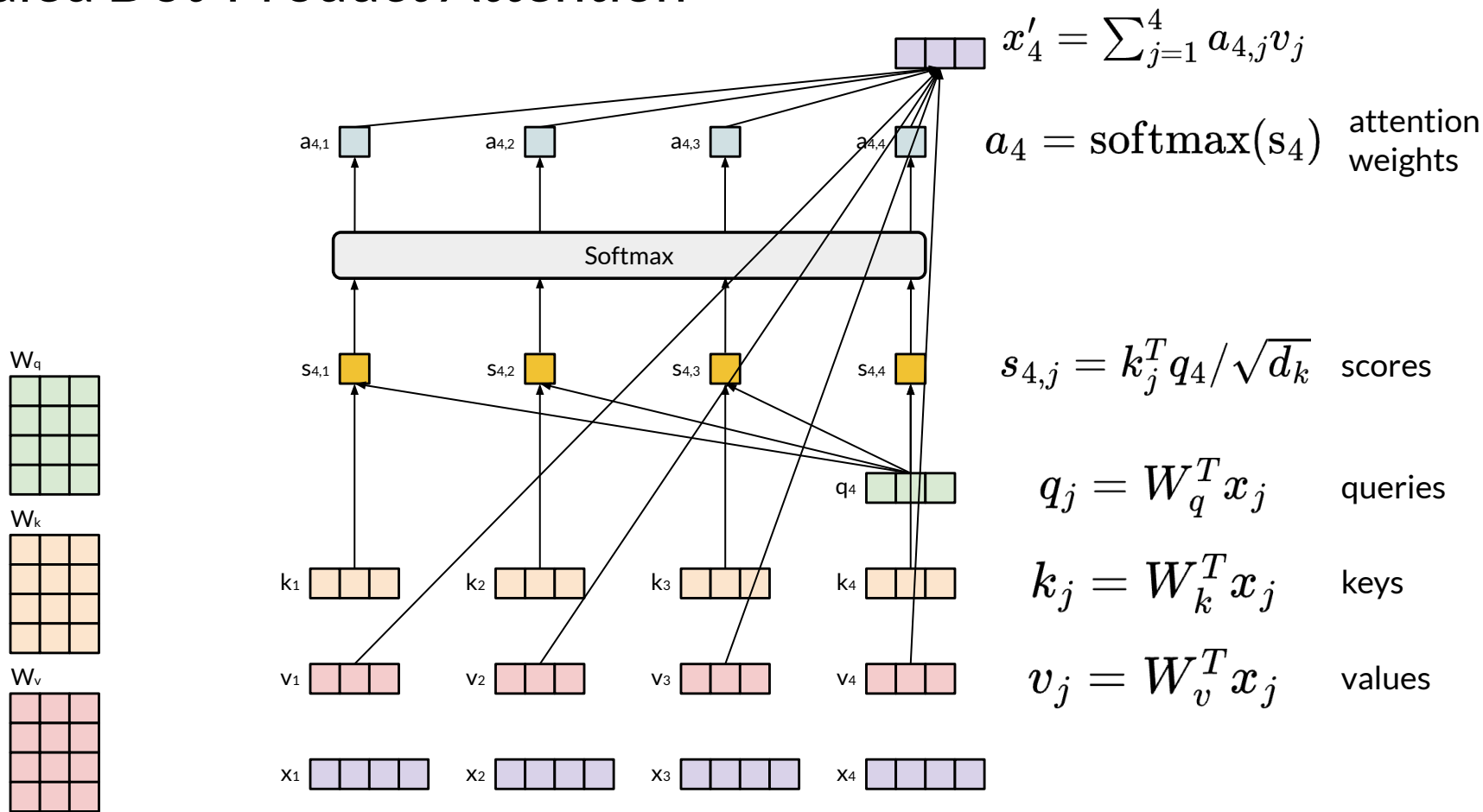
Scaled Dot-Product Attention



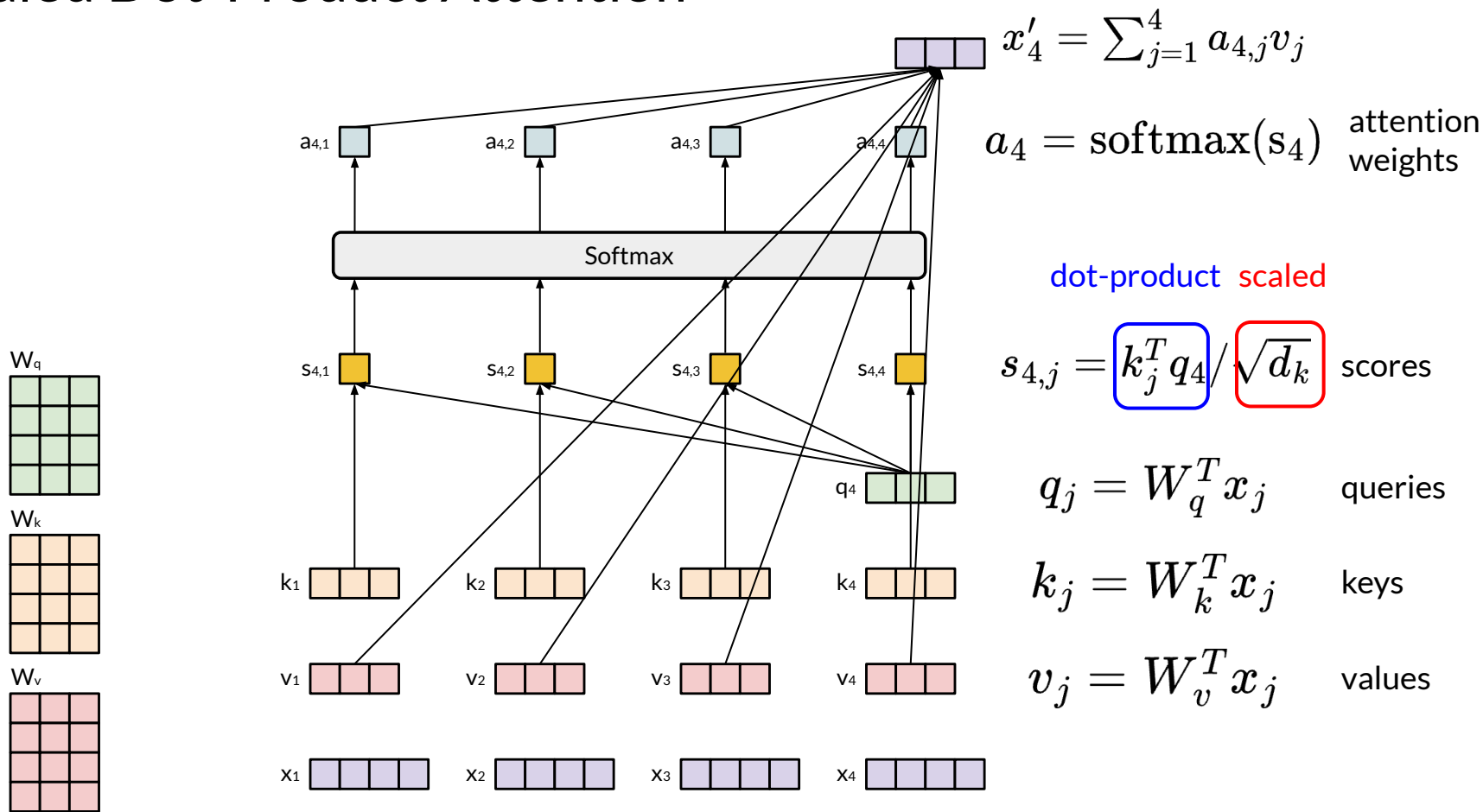
Scaled Dot-Product Attention



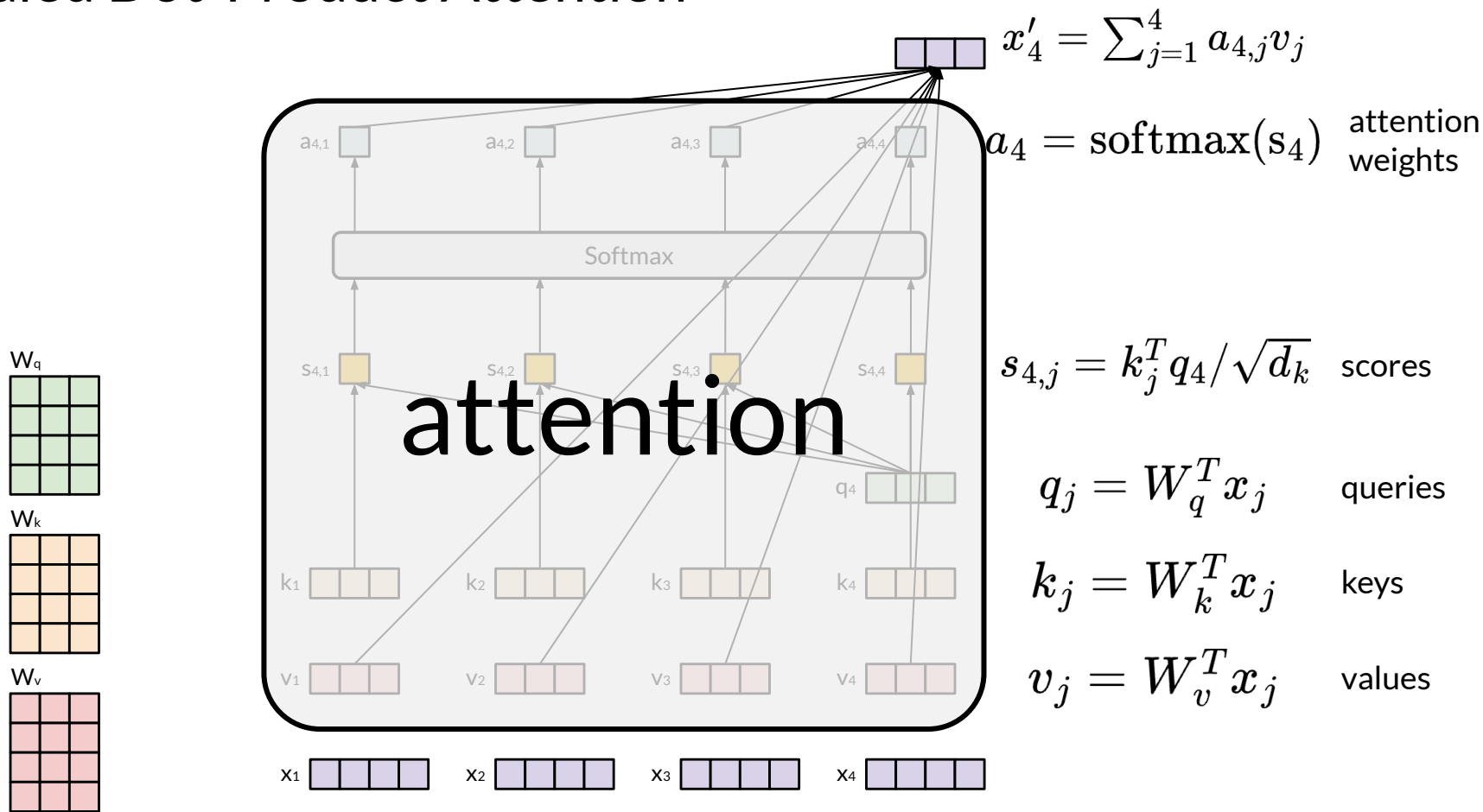
Scaled Dot-Product Attention



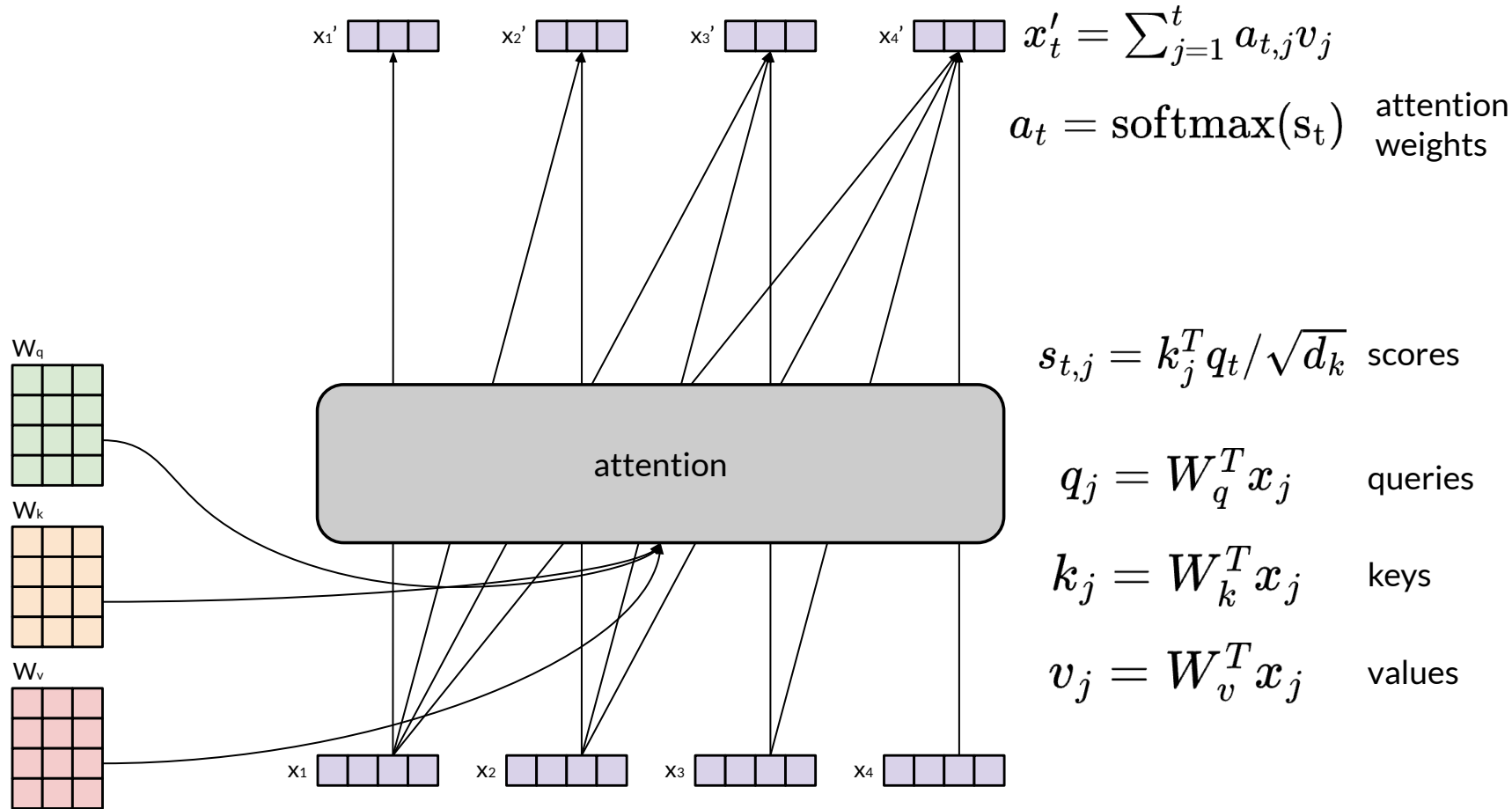
Scaled Dot-Product Attention



Scaled Dot-Product Attention



Scaled Dot-Product Attention



Multi-head Attention

- Multi-head attention applies multiple attention heads in parallel with different learned projections of Q, K, and V, allowing each head to capture different relationships.
- Example: The animal didn't cross the street because it was too ?
 - Head 1: attends to "animal" -> *tired*
 - Head 2: attends to "street" -> *wide*

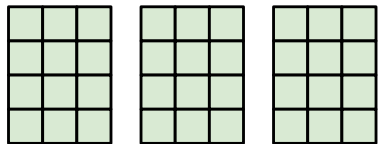
Multi-head Attention

1st
attn.
head

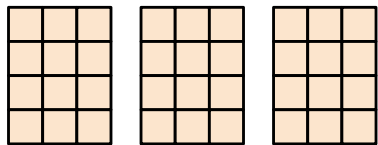
2nd
attn.
head

3rd
attn.
head

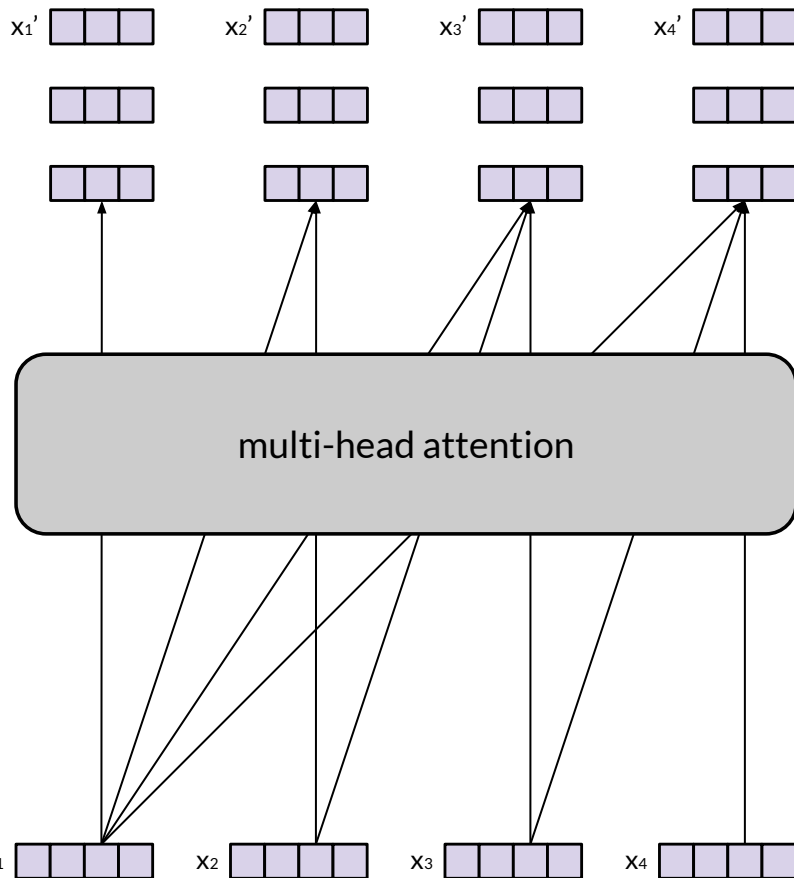
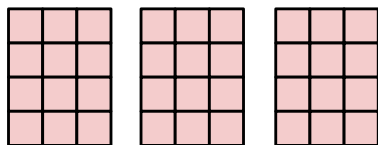
W_q



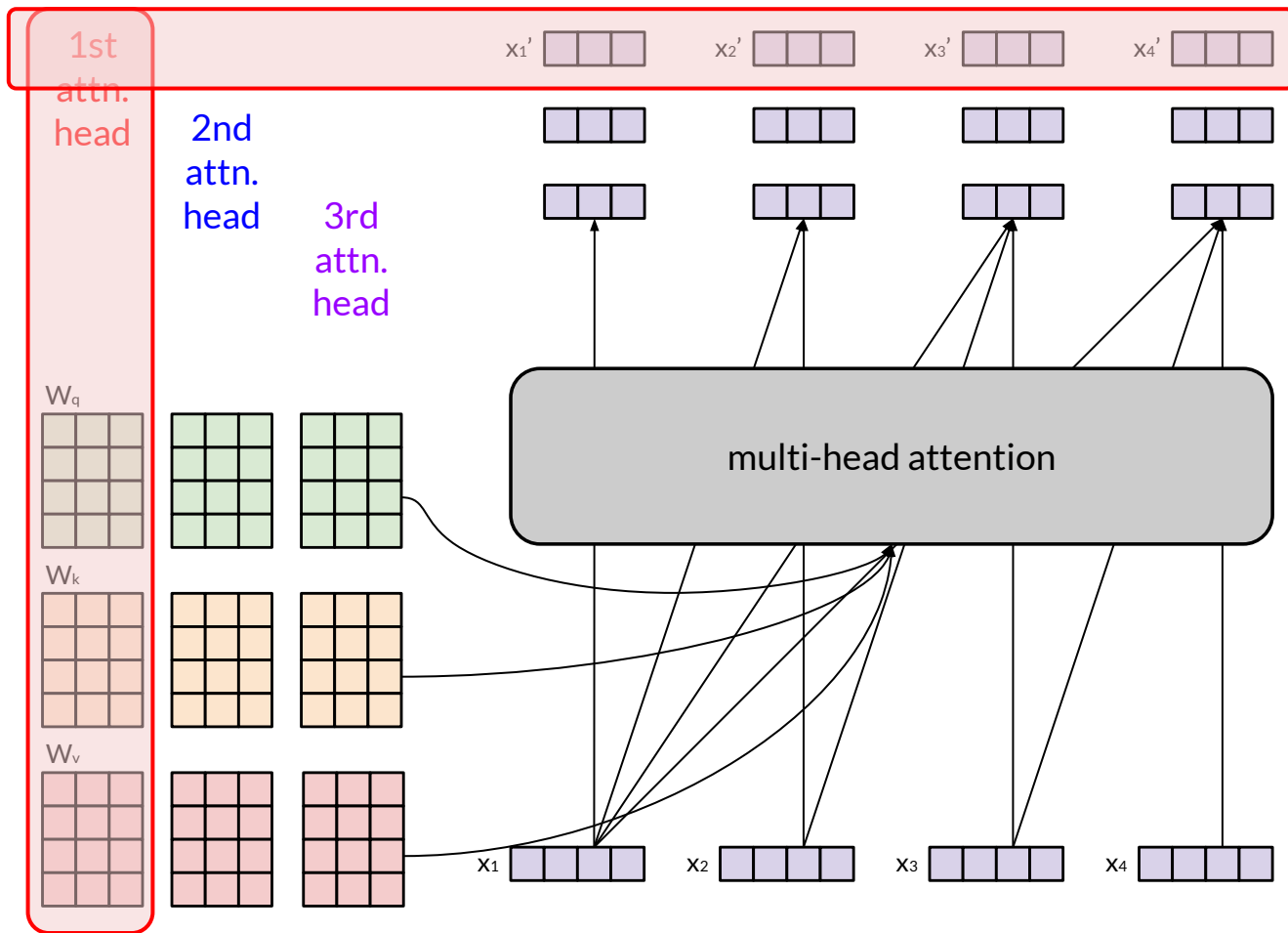
W_k



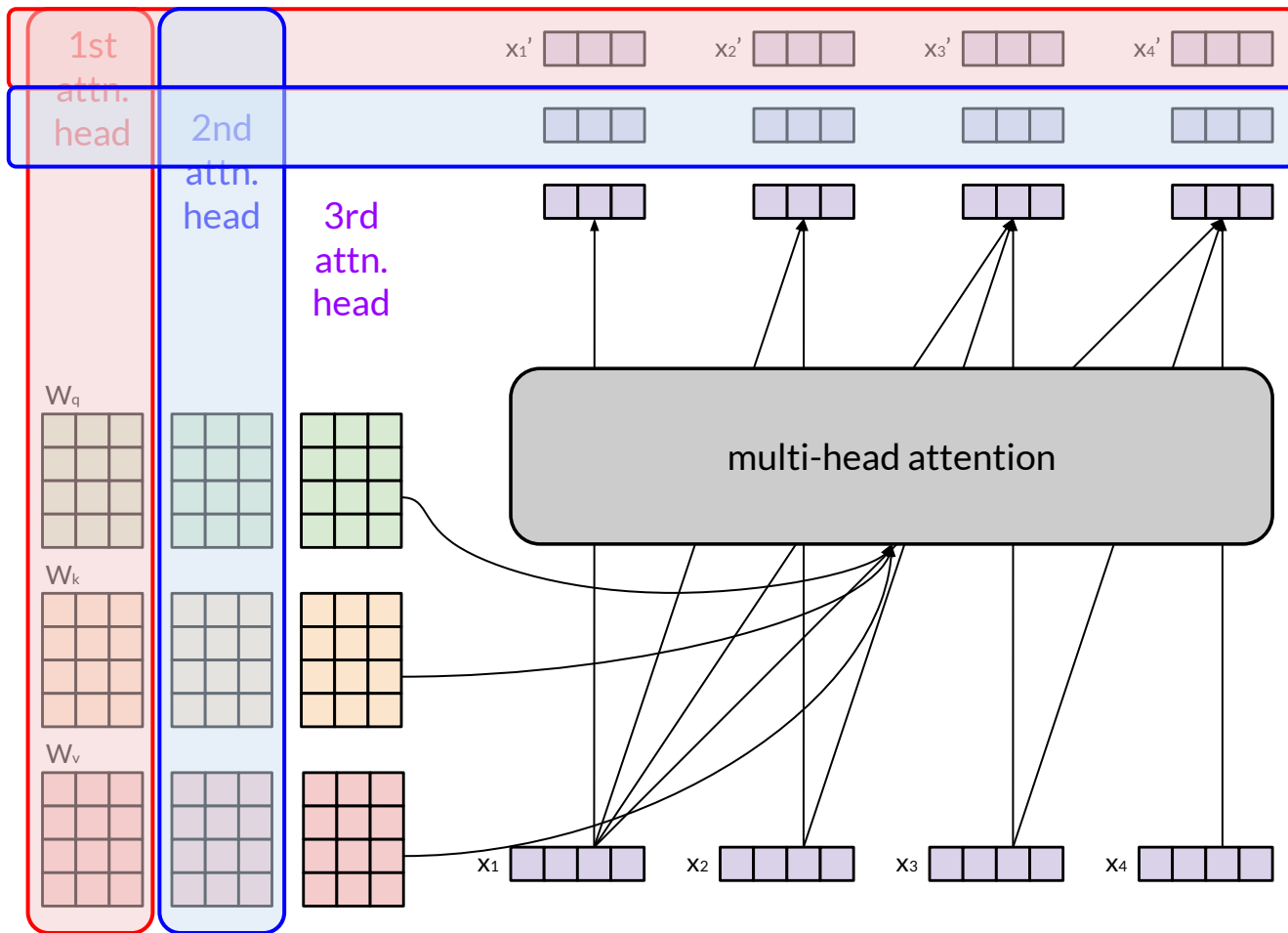
W_v



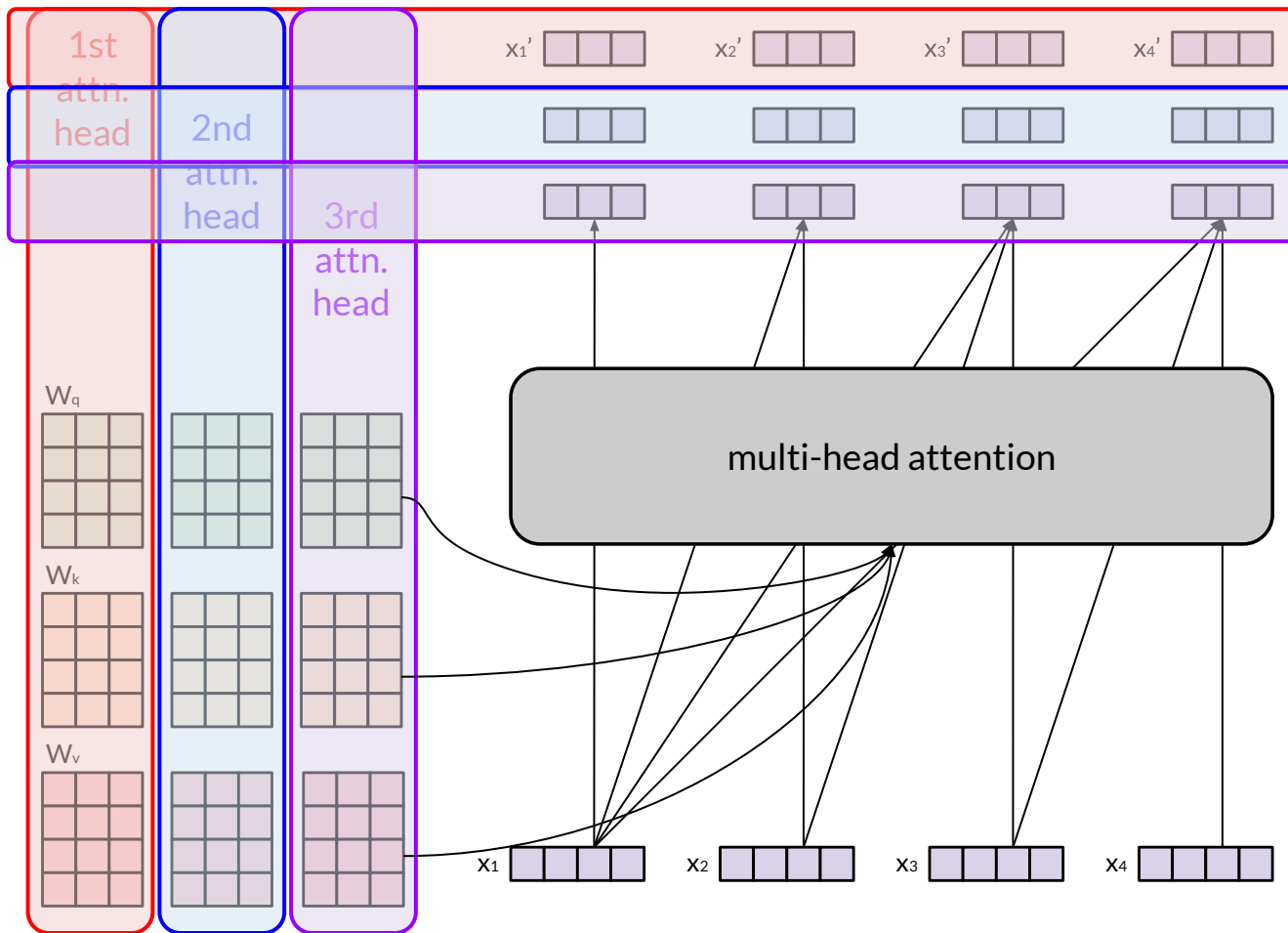
Multi-head Attention



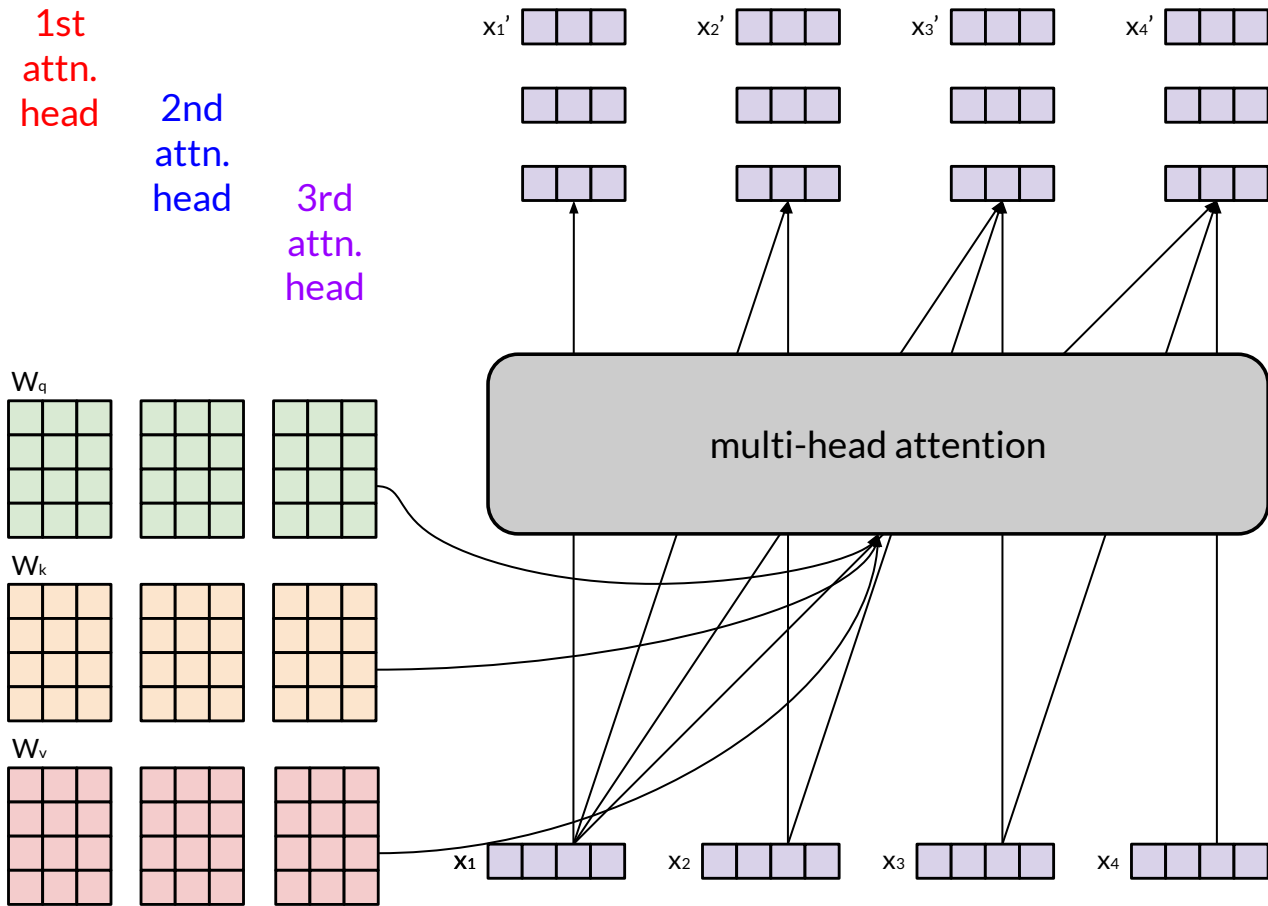
Multi-head Attention



Multi-head Attention

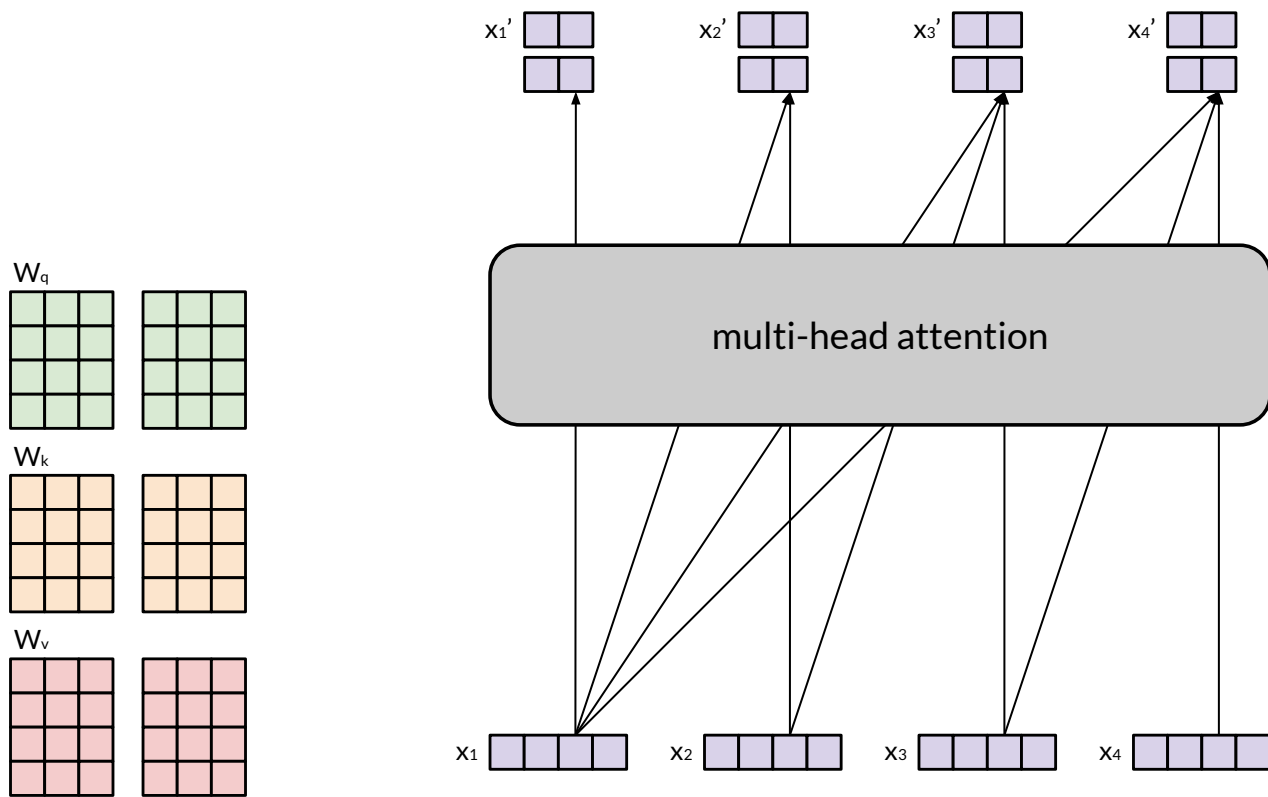


Multi-head Attention



- One attention layer, multiple channels (i.e., multi-head).
- Each head has its own parameters.
- Then concatenate them.

Multi-head Attention



- One attention layer, multiple channels (i.e., multi-head).
- Each head has its own parameters.
- Then concatenate them.
- Ensure the dimension of input and output is consistent, we should choose the embedding sizes and number of heads appropriately:
 - input dim = 4
 - head num = 2
 - output dim = input dim / head num = 2

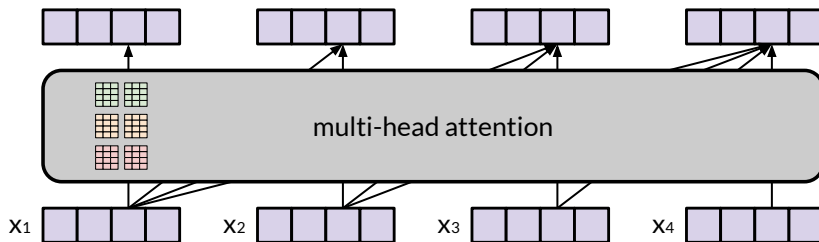
Layer Normalization

- Layer normalization normalizes the features of each token's hidden representation to stabilize training.
- Problem: *internal covariate shift* occurs during training of a deep network when a small change in the low layers amplifies into a large change in the high layers.
- Formula: $\text{LayerNorm}(x) = \gamma \odot (x - \mu) / \sigma \oplus \beta$
 - γ and β are trainable parameters.
 - μ and σ are mean and standard deviation.
 - $\odot$ and $\oplus$ are elementwise multiplication and addition.

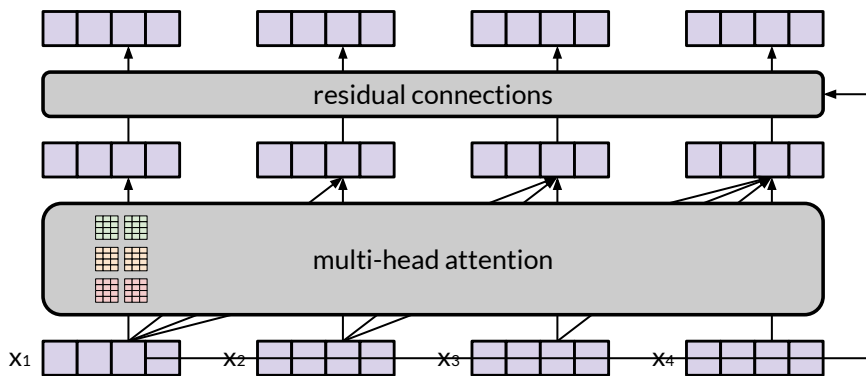
Residual Connections

- Residual connections add a layer's input to its output, allowing information and gradients to flow more directly through deep networks.
- Problem: As neural networks become deeper, they can become harder to optimize, leading to performance degradation.
- Formula: $y = x + F(x)$
- Instead of $F(x)$ having to learn a full transformation of x , $F(x)$ only needs to learn an additive modification of x (i.e. the residual).

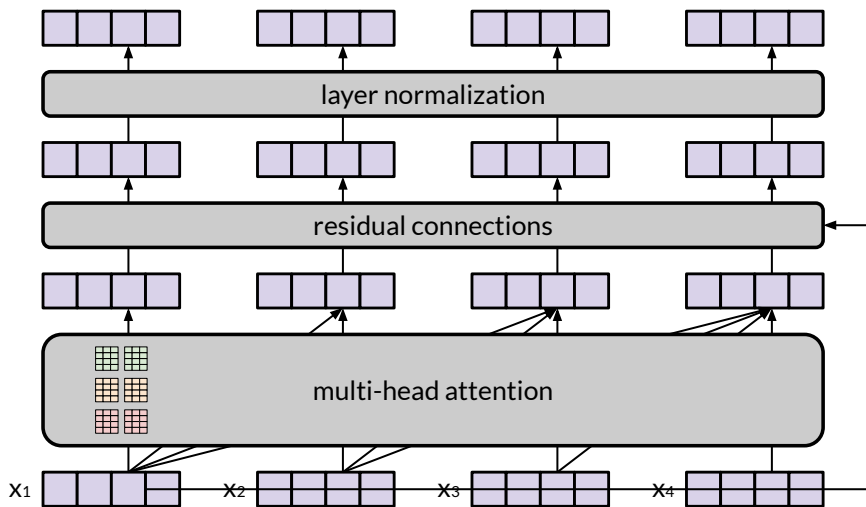
Transformer Layer



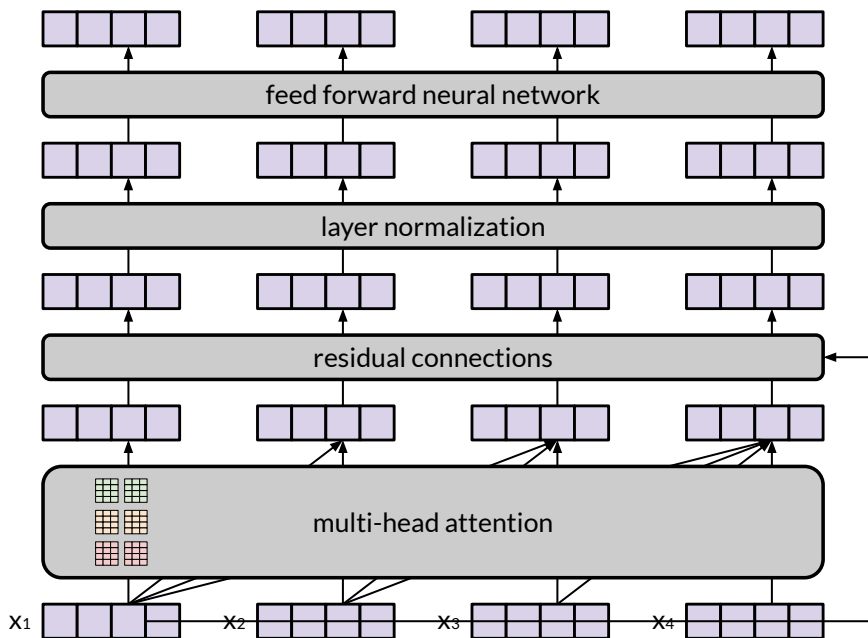
Transformer Layer



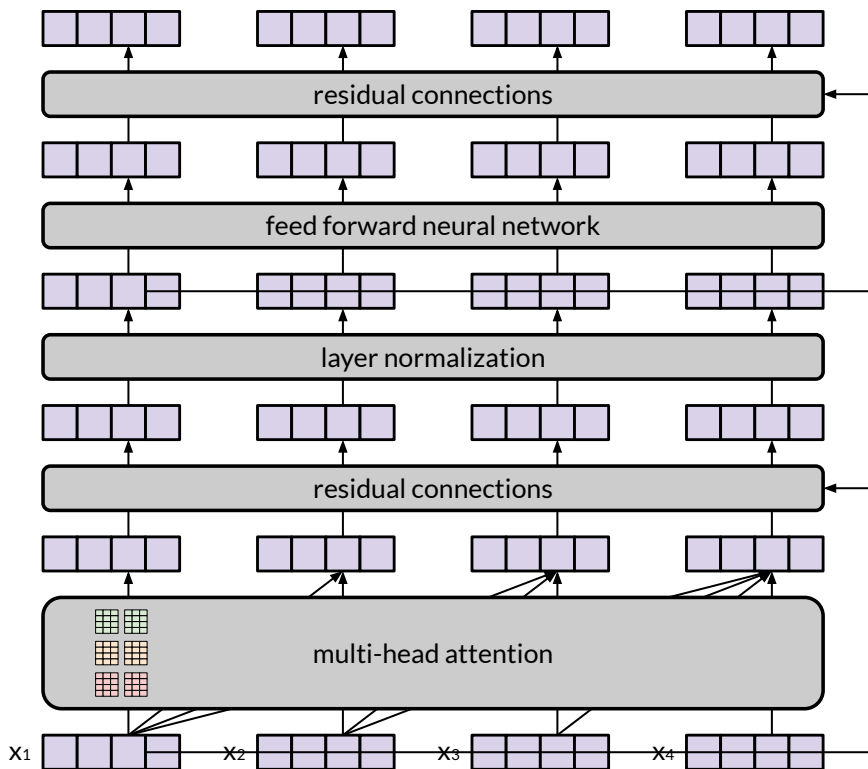
Transformer Layer



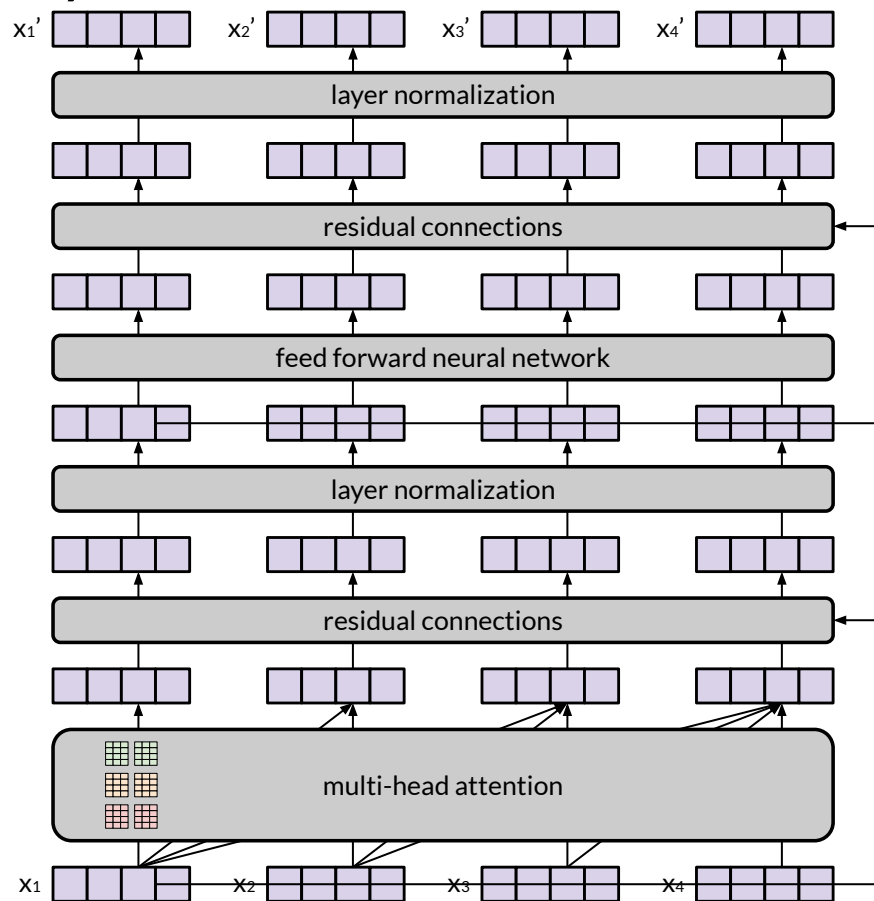
Transformer Layer



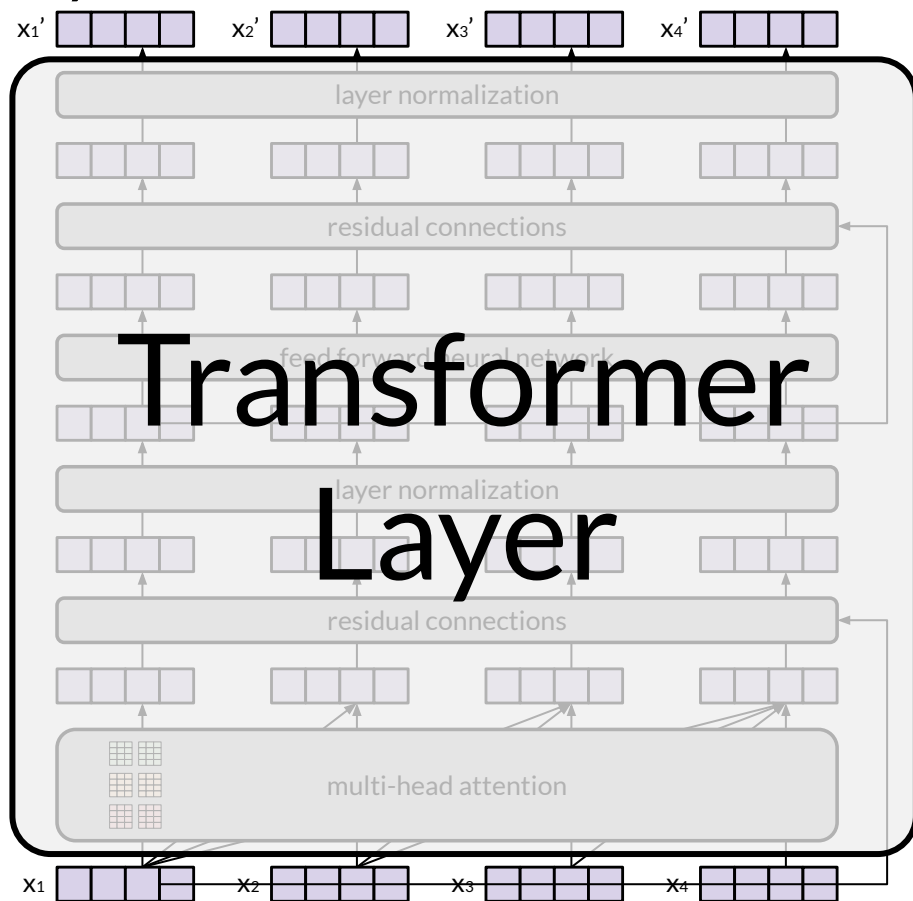
Transformer Layer



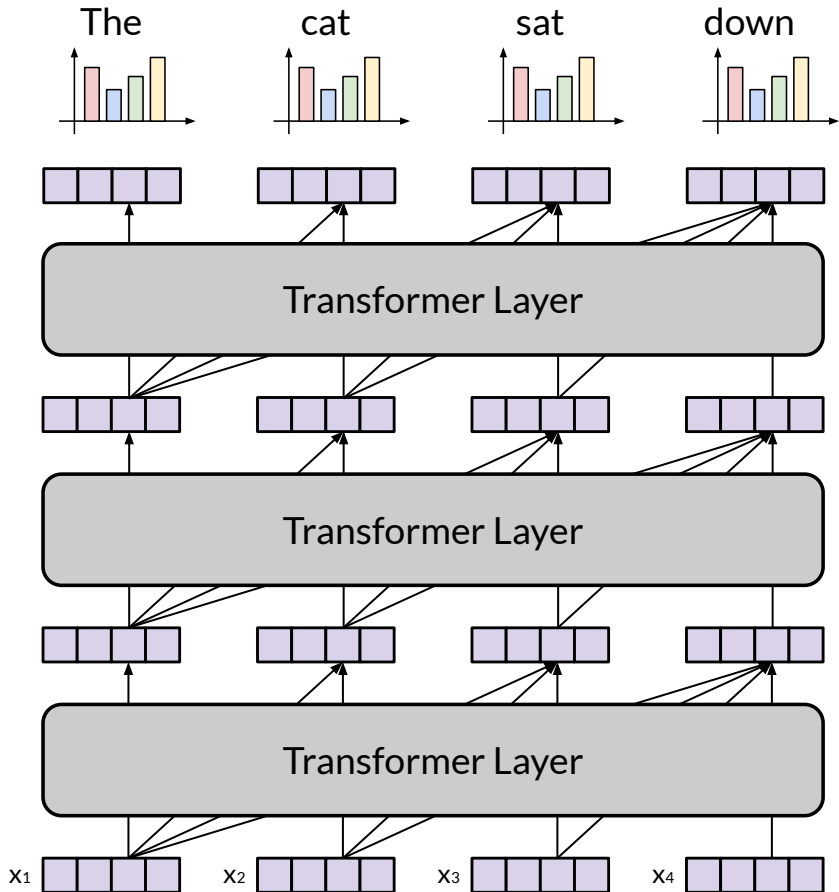
Transformer Layer



Transformer Layer

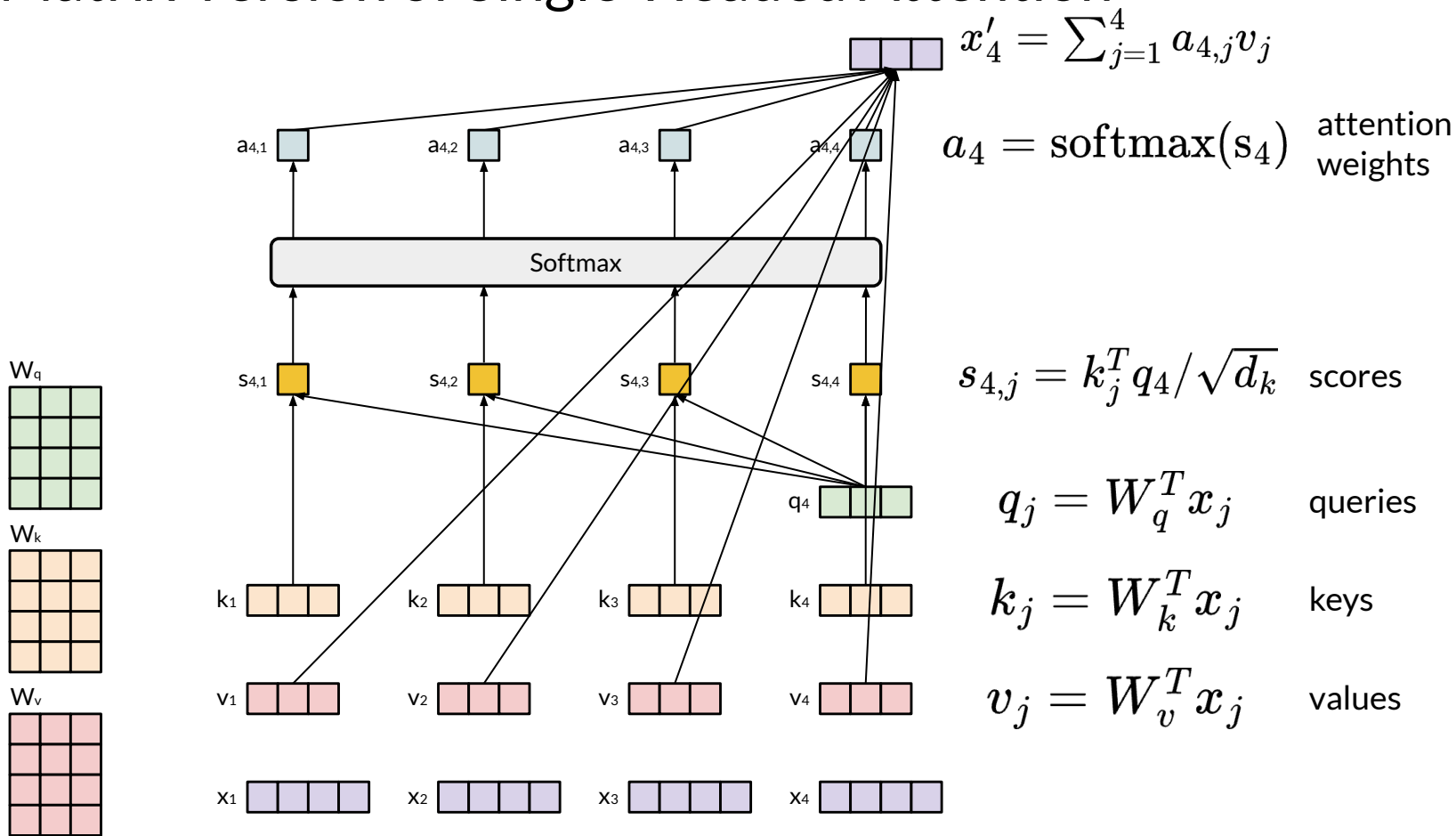


Transformer Language Model

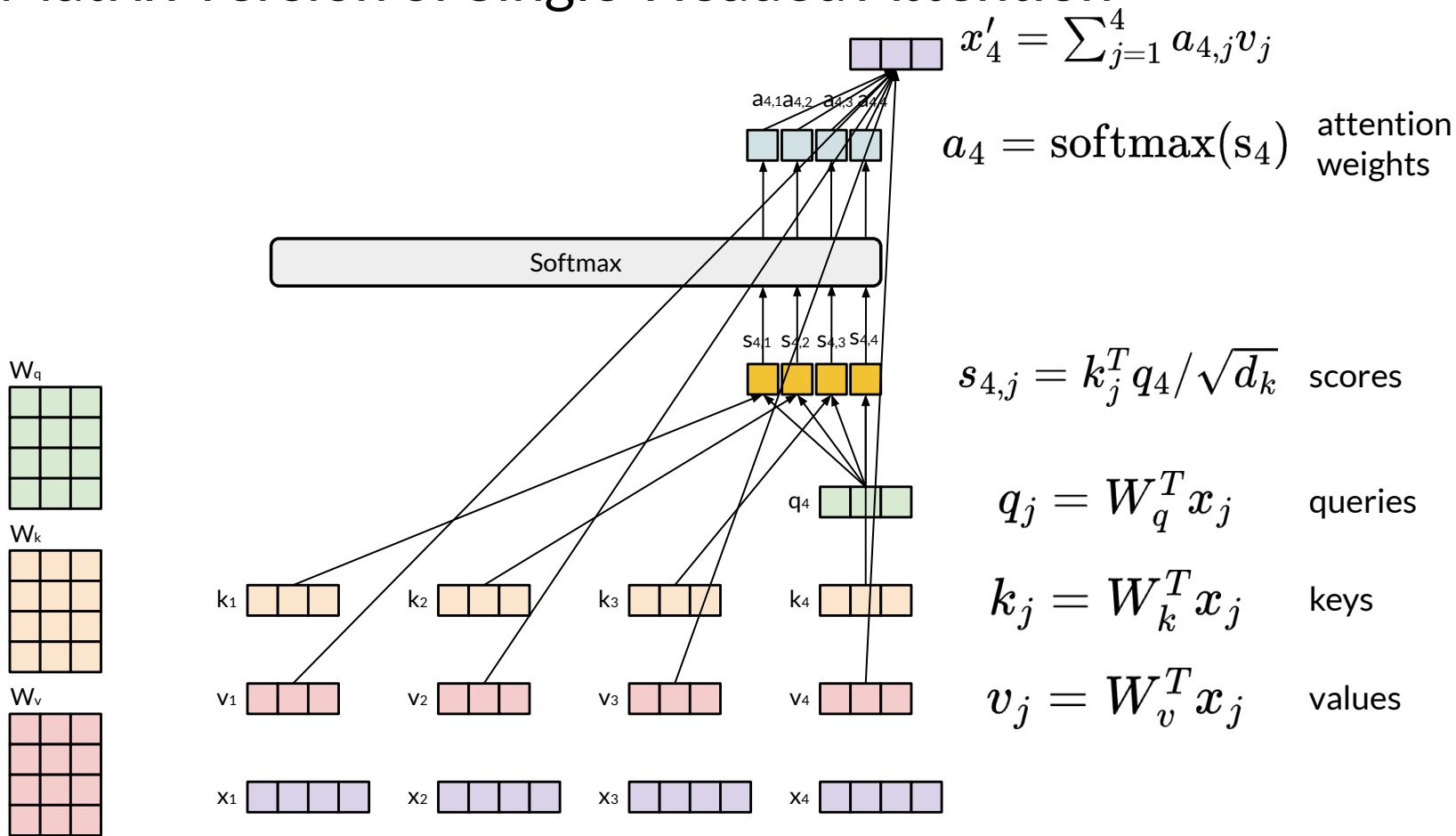


A small Phi-3-mini-instruct (3.8B parameter) model has 32 Transformer layers.

Matrix Version of Single-Headed Attention

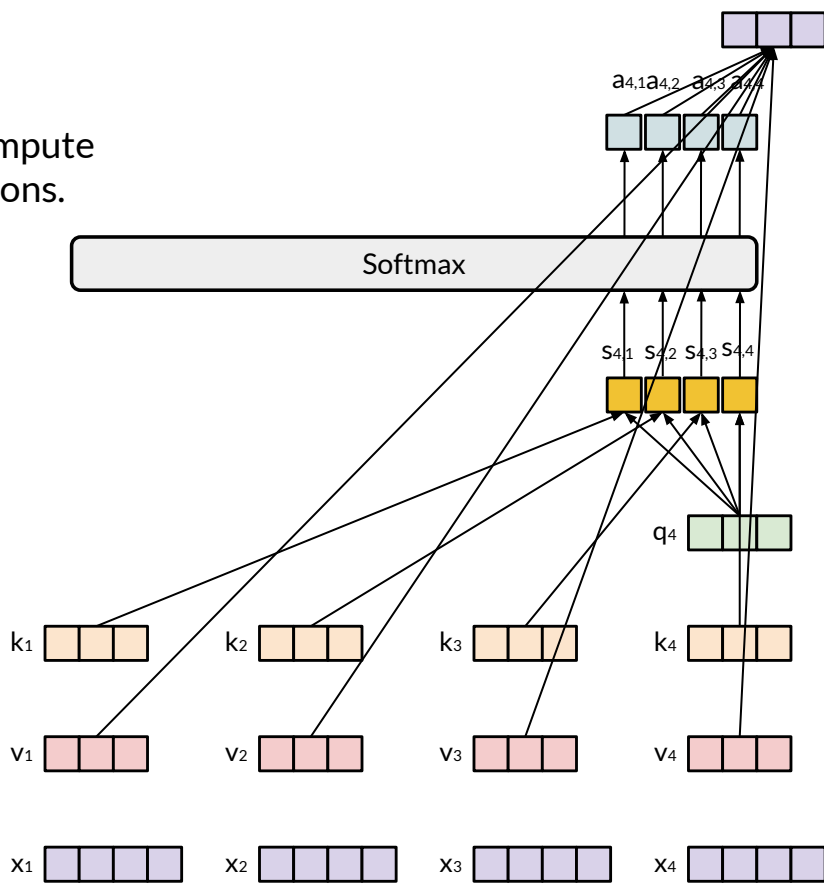
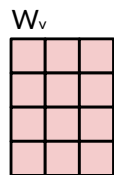
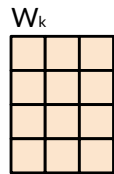
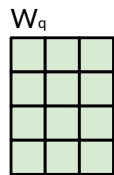


Matrix Version of Single-Headed Attention



Matrix Version of Single-Headed Attention

For speed, we compute
by matrix operations.



$$X' = AV = \text{softmax}(QK^T / \sqrt{d_k})V$$

$$A = [a_1, \dots, a_4]^T = \text{softmax}(S)$$

$$S = [s_1, \dots, s_4]^T = QK^T / \sqrt{d_k}$$

$$Q = [q_1, \dots, q_4]^T = XW_q$$

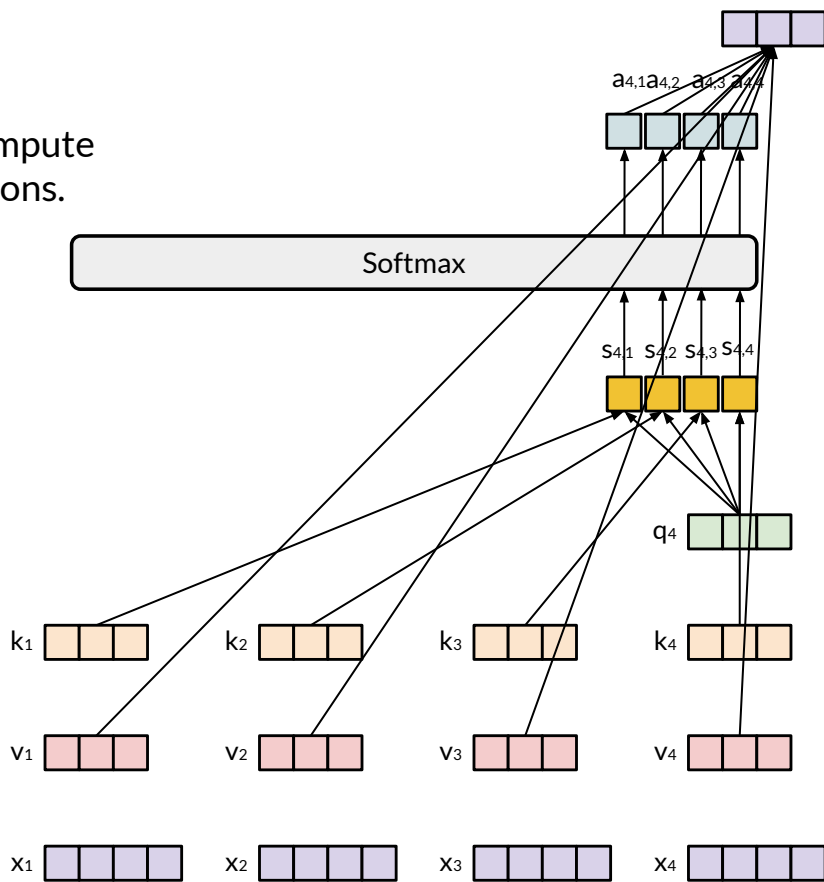
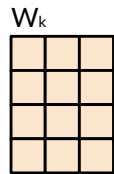
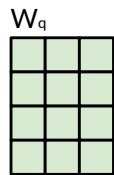
$$K = [k_1, \dots, k_4]^T = XW_k$$

$$V = [v_1, \dots, v_4]^T = XW_v$$

$$X = [x_1, \dots, x_4]^T$$

Matrix Version of Single-Headed Attention

For speed, we compute
by matrix operations.



$$X' = AV = \text{softmax}(QK^T / \sqrt{d_k})V$$

$$A = [a_1, \dots, a_4]^T = \text{softmax}(S)$$

$$\mathbb{R}^{N \times N}$$

$$S = [s_1, \dots, s_4]^T = QK^T / \sqrt{d_k} \quad \mathbb{R}^{N \times N}$$

$$Q = [q_1, \dots, q_4]^T = XW_q \quad \mathbb{R}^{N \times d_k}$$

$$K = [k_1, \dots, k_4]^T = XW_k \quad \mathbb{R}^{N \times d_k}$$

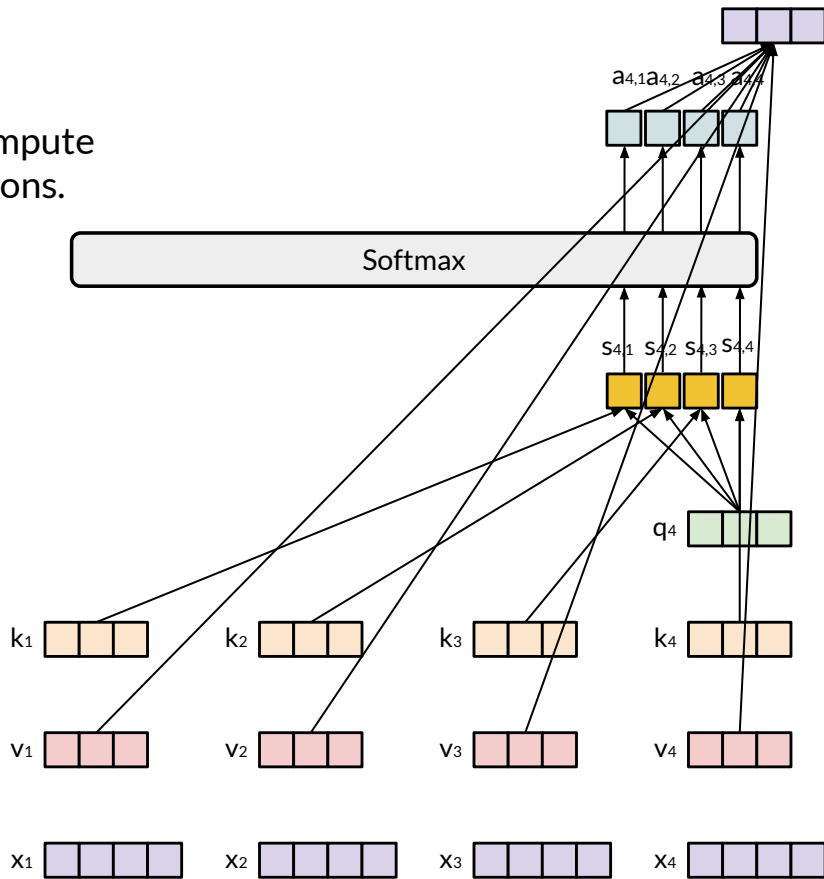
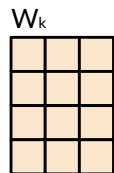
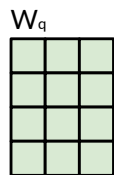
$$V = [v_1, \dots, v_4]^T = XW_v \quad \mathbb{R}^{N \times d_k}$$

$$X = [x_1, \dots, x_4]^T \quad \mathbb{R}^{N \times d_{\text{model}}}$$

Matrix Version of Single-Headed Casual Attention

$$\mathbb{R}^{N \times d_k}$$

For speed, we compute by matrix operations.



$$X' = AV = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

$$A_{\text{casual}} = \text{softmax}(S + M)$$

$$\mathbb{R}^{N \times N}$$

$$S = [s_1, \dots, s_4]^T = QK^T / \sqrt{d_k} \mathbb{R}^{N \times N}$$

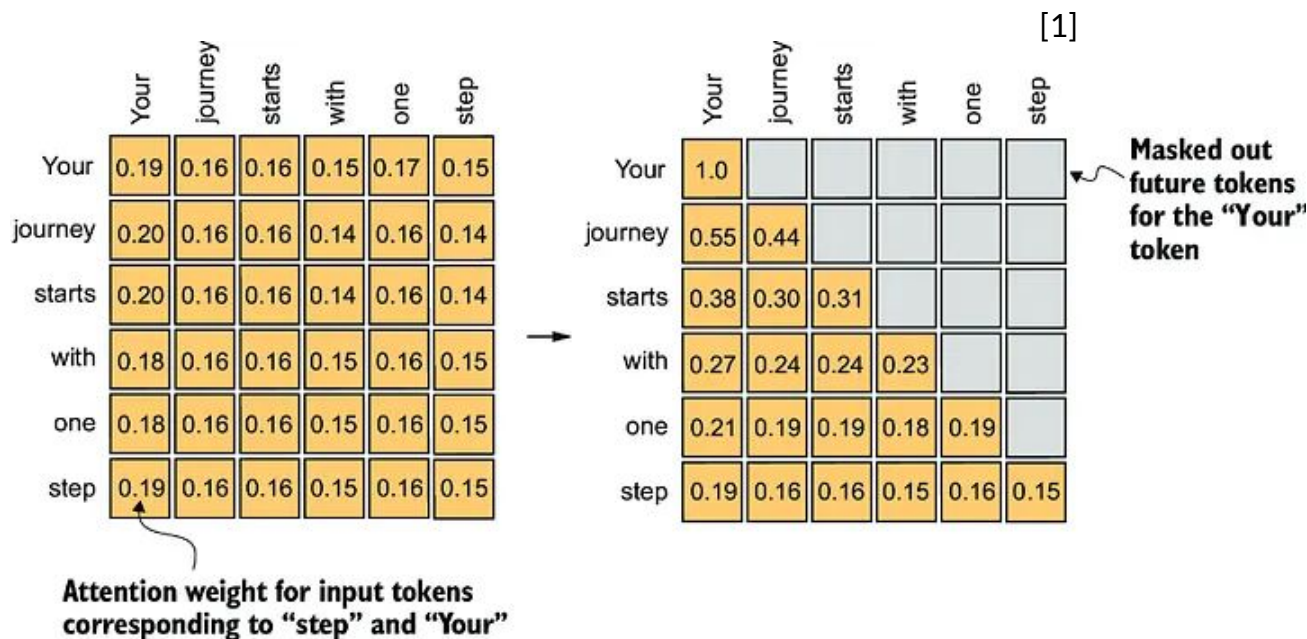
$$Q = [q_1, \dots, q_4]^T = XW_q \mathbb{R}^{N \times d_k}$$

$$K = [k_1, \dots, k_4]^T = XW_k \mathbb{R}^{N \times d_k}$$

$$V = [v_1, \dots, v_4]^T = XW_v \mathbb{R}^{N \times d_k}$$

$$X = [x_1, \dots, x_4]^T \mathbb{R}^{N \times d_{\text{model}}}$$

Illustrating Casual Attention Mask



Acknowledgement

- Parts of these slides were adapted from and inspired by materials from CMU 10-423/10-623/10-723: Generative AI, particularly the [lecture on Transformer Language Models](#). I gratefully acknowledge the course instructors and authors of these materials.